fetel@HCMUS
KHOA ĐIỆN TỬ - VIỄN THÔNG

DESLAB

BÁO CÁO HÀNG TUẦN

Project: Klein Module

Name: Bùi Quốc Hưng
Position: K23_CLC
Date: 18/04/2026

1. Nội dung báo cáo hàng tuần

Thời gian vừa qua (tuần vừa qua):

+ Đã thực hiện tối ưu DataPath: Ghép 2 module Cipher và Decipher thành 1 Path
+ Đã thực hiện tối ưu Key Schedule: Tối ưu cho Encryption và Decryption
+ Đã thực hiện tối ưu Core: Thêm Caches đệm cho Key 0th và Key 12th
+ Đang tiến hành Nạp xuống board DE2: So sánh tối ưu tài nguyên

Thời gian tới (tuần tới):

+ Kế hoạch sắp tới: Đọc về FIPS 205, Tiếp tục việc Nạp xuống board

2. Structure of KLEIN

The structure of KLEIN is a typical Substitution-Permutation Network (SPN), which is also used in many advanced block ciphers, e.g. AES and PRESENT. In our first estimation for obtaining a reasonable security margin and asymmetric iteration, we choose the number of rounds N_R as 12/16/20 for KLEIN-64/80/96 respectively. A high-level description of the KLEIN encryption routine is described in Figure 1.

$sk^1 \leftarrow$ KEY;
STATE $\leftarrow$ PLAINTEXT;
for $i = 1$ to N_R do
  $AddRoundKey$(STATE, sk^i);
  $SubNibbles$(STATE);
  $RotateNibbles$(STATE);
  $MixNibbles$(STATE);
  $sk^{i+1} = KeySchedule(sk^i, i)$;
end for
CIPHERTEXT $\leftarrow$ $AddRoundKey$(STATE, sk^{N_R+1});

Figure 1: The encryption routine of KLEIN.

2. Structure of KLEIN

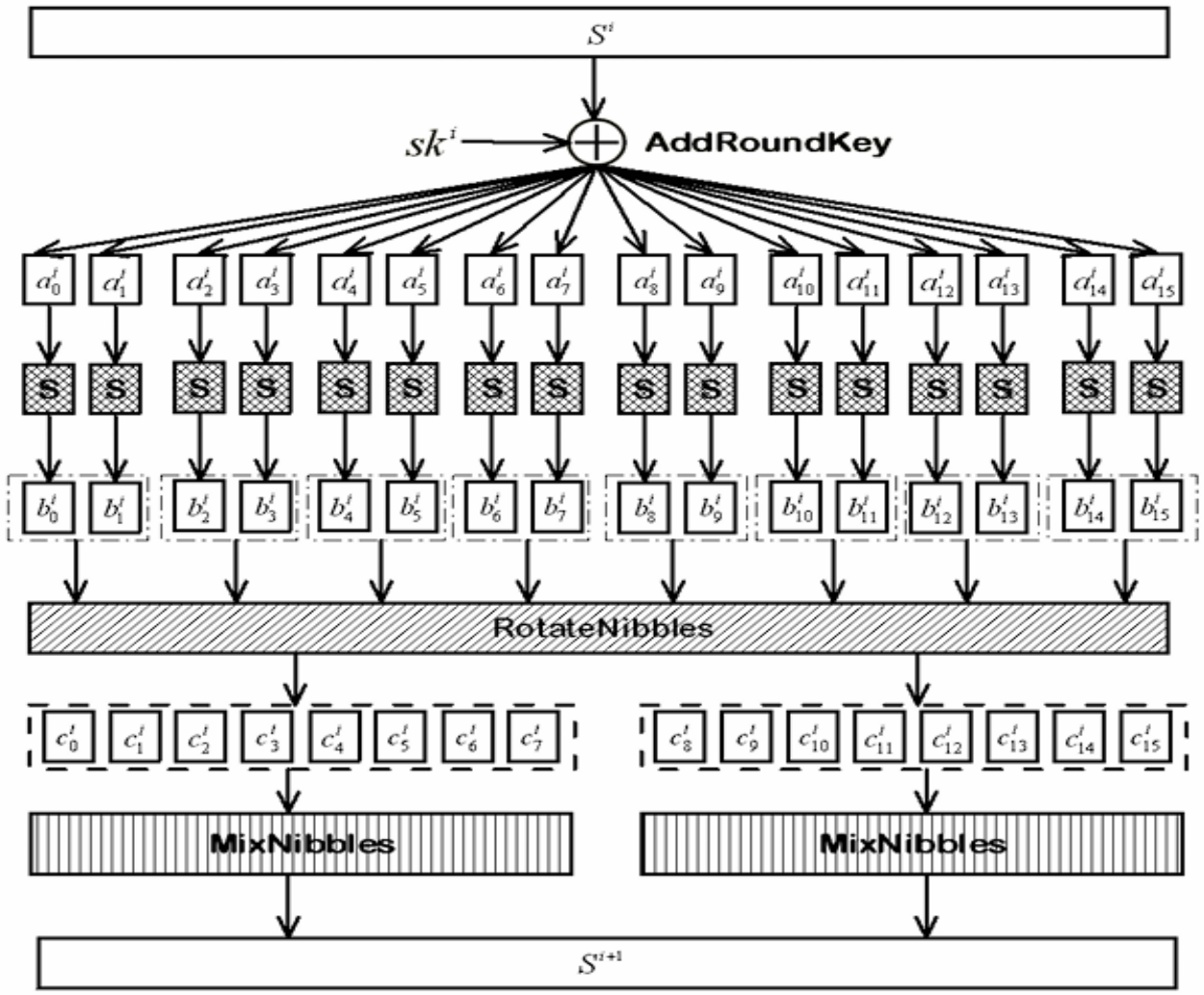


Figure 2: The round transformation of KLEIN.

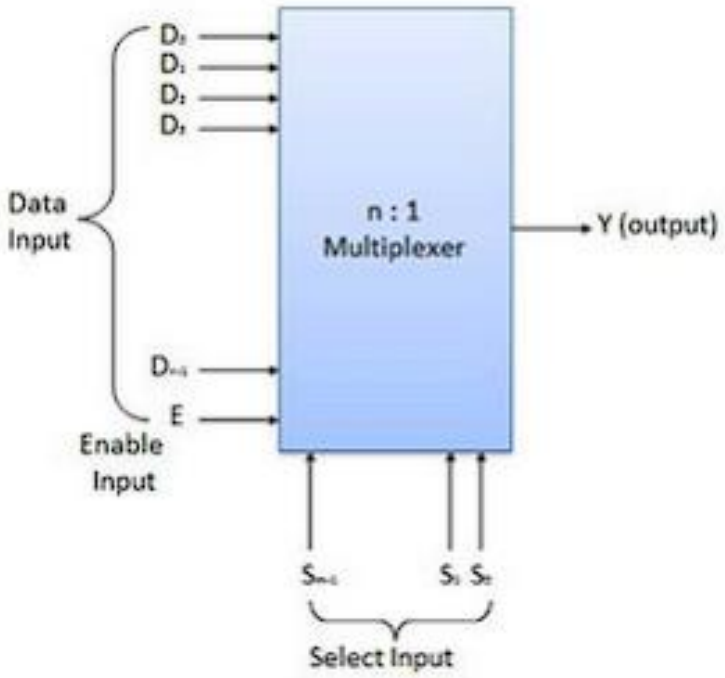
2. Structure of KLEIN (Sbox)

3.2.1 The SubNibbles Step

Before the SubNibbles step, the input text will be xored with the i-th round key sk^i, where $i \in [1, N_R]$. In the SubNibbles step, the xored results will be divided into 16 of 4-bit nibbles and input to the same 16 S-boxes. The KLEIN S-box S is a 4×4 involutive permutation. The non-linear permutation executed by S is described in Table 1. The implementation costs of such

Table 1: The 4-Bit S-box used in KLEIN.

Input	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
Output	7	4	A	9	1	F	B	0	C	3	2	6	8	E	D	5




```
module Klein_Sbox(in, out);
  input wire [3:0] in;
  output wire [3:0] out;

  reg [3:0] reg_out;

  assign out = reg_out;

  always @(*) begin
   case (in)
    4'h0: reg_out = 4'h7;
    4'h1: reg_out = 4'h4;
    4'h2: reg_out = 4'hA;
    4'h3: reg_out = 4'h9;
    4'h4: reg_out = 4'h1;
    4'h5: reg_out = 4'hF;
    4'h6: reg_out = 4'hB;
    4'h7: reg_out = 4'h0;
    4'h8: reg_out = 4'hC;
    4'h9: reg_out = 4'h3;
    4'hA: reg_out = 4'h2;
    4'hB: reg_out = 4'h6;
    4'hC: reg_out = 4'h8;
    4'hD: reg_out = 4'hE;
    4'hE: reg_out = 4'hD;
    4'hF: reg_out = 4'h5;
   default: reg_out = 4'h5;
  endcase
 end

endmodule
```

2. Structure of KLEIN (Mix Nibbles)

3.2.2 The RotateNibbles Steps

During the i-th round where $i \in [1, N_R]$, 16 nibbles $b_0^i, b_1^i, \cdots, b_{15}^i$ will be rotated left two bytes per round, which is illustrated in Figure 3. The inverse operation will be simply rotate right two bytes per round. Nevertheless, the RotateNibbles step can also be combined with the MixNibbles step to avoid the hardware or software costs.

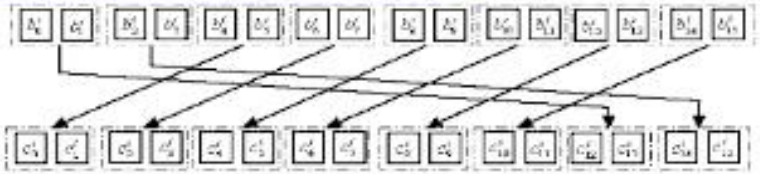


Figure 3: The RotateNibbles step.

3.2.3 The MixNibbles Step

The MixNibbles step is a bricklayer permutation of the state. The i-th round input nibbles $\{c_0^i, c_1^i, \cdots, c_{15}^i\}$ will be divided into 2 tuples, which will be proceeded the same as the MixColumns step in Rijndael. The tuples of the state are considered as polynomials over $\mathbb{F}_2^8$ and multiplied modulo $x^4 + 1$ with a fixed polynomial $c(x) = 03 \cdot x^3 + 01 \cdot x^2 + 01 \cdot x + 02$. The inverse is also a fixed multiplication polynomial $d(x) = 0B \cdot x^3 + 0D \cdot x^2 + 09 \cdot x + 0E$. The output of the MixNibbles step will be the intermediate state s^{i+1} for the next round transformation, which can be represented by the following matrix equations,

$$\begin{bmatrix} s_0^{i+1}||s_1^{i+1} \\ s_2^{i+1}||s_3^{i+1} \\ s_4^{i+1}||s_5^{i+1} \\ s_6^{i+1}||s_7^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_0^i||c_1^i \\ c_2^i||c_3^i \\ c_4^i||c_5^i \\ c_6^i||c_7^i \end{bmatrix}, \begin{bmatrix} s_8^{i+1}||s_9^{i+1} \\ s_{10}^{i+1}||s_{11}^{i+1} \\ s_{12}^{i+1}||s_{13}^{i+1} \\ s_{14}^{i+1}||s_{15}^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_8^i||c_9^i \\ c_{10}^i||c_{11}^i \\ c_{12}^i||c_{13}^i \\ c_{14}^i||c_{15}^i \end{bmatrix}.$$

Note that the balance between the diffusion property and the software performance let us make this choice. Although bit-shifting operations are often used in the diffusion layer of many lightweight block ciphers (e.g. PRESENT and NOEKEON), it actually loses the efficiency in software implementations. From the number of active Sboxes, it seems a better choice that the MixNibbles step chooses a matrix multiplication in $GF(2^4)$. However, a byte-oriented matrix multiplication has advantage in the software implementations for 8-bit processors (e.g. Skipjack). By using the similar implementation of the MixColumns step for 8-bit processors [11], we can use just one 256-byte look-up table to optimize the MixNibbles step. Also the same look-up table can be used to optimize its inverse. After the 12/16/20 rounds of KLEIN-64/80/96, the MixNibbles step can still provide a high number of active Sboxes for the security of KLEIN. The property of the MixColumns step of Rijndael has been well-analyzed, the details can be found in the literature [9, 11, 13].

$$\begin{bmatrix} s_0^{i+1}||s_1^{i+1} \\ s_2^{i+1}||s_3^{i+1} \\ s_4^{i+1}||s_5^{i+1} \\ s_6^{i+1}||s_7^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_0^i||c_1^i \\ c_2^i||c_3^i \\ c_4^i||c_5^i \\ c_6^i||c_7^i \end{bmatrix}, \begin{bmatrix} s_8^{i+1}||s_9^{i+1} \\ s_{10}^{i+1}||s_{11}^{i+1} \\ s_{12}^{i+1}||s_{13}^{i+1} \\ s_{14}^{i+1}||s_{15}^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_8^i||c_9^i \\ c_{10}^i||c_{11}^i \\ c_{12}^i||c_{13}^i \\ c_{14}^i||c_{15}^i \end{bmatrix}.$$

Phép MixColumns

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

Kết quả cho cột 1

({02}.{87}) ⊕ ({03}.{6E}) ⊕ {46} ⊕ {A6} = {47}

{87} ⊕ ({02} .{6E}) ⊕ ({03}.{46}) ⊕ {A6} = {37}

{87} ⊕ {6E} ⊕ ({02}.{46}) ⊕ ({03}.{A6}) = {94}

({03} .{87}) ⊕ {6E} ⊕ {46} ⊕ ({02}.{A6}) = {ED}

2. Structure of KLEIN (Mix Nibbles)

$$\begin{bmatrix} s_0^{i+1}||s_1^{i+1} \\ s_2^{i+1}||s_3^{i+1} \\ s_4^{i+1}||s_5^{i+1} \\ s_6^{i+1}||s_7^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_0^i||c_1^i \\ c_2^i||c_3^i \\ c_4^i||c_5^i \\ c_6^i||c_7^i \end{bmatrix}, \begin{bmatrix} s_8^{i+1}||s_9^{i+1} \\ s_{10}^{i+1}||s_{11}^{i+1} \\ s_{12}^{i+1}||s_{13}^{i+1} \\ s_{14}^{i+1}||s_{15}^{i+1} \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \times \begin{bmatrix} c_8^i||c_9^i \\ c_{10}^i||c_{11}^i \\ c_{12}^i||c_{13}^i \\ c_{14}^i||c_{15}^i \end{bmatrix}.$$

```
////////////////////////////////////////////////////////////////////////////////
function [7 : 0] gm2(input [7 : 0] op);
  begin
    gm2 = {op[6:4], op[3]^op[7], op[2]^op[7], op[1], op[0]^op[7], op[7]};
  end
endfunction // gm2
////////////////////////////////////////////////////////////////////////////////
```

```
////////////////////////////////////////////
wire [7:0] sum = w0 ^ w1 ^ w2 ^ w3;

assign a0 = gm2(w0 ^ w1) ^ sum ^ w0;
assign a1 = gm2(w1 ^ w2) ^ sum ^ w1;
assign a2 = gm2(w2 ^ w3) ^ sum ^ w2;
assign a3 = gm2(w3 ^ w0) ^ sum ^ w3;
////////////////////////////////////////


assign b0 = gm4(a0 ^ a2) ^ a0;
assign b1 = gm4(a1 ^ a3) ^ a1;
assign b2 = gm4(a0 ^ a2) ^ a2;
assign b3 = gm4(a1 ^ a3) ^ a3;

assign odata = (iinv) ? {b0, b1, b2, b3} : {a0, a1, a2, a3};
```

```
module Klein_Mixnibbles(idata, iinv, odata);
    input       wire    [31:0]   idata;
    input       wire             iinv;
    output      wire    [31:0]   odata;


////////////////////////////////////////////////////////////////////////////////
function [7 : 0] gm2(input [7 : 0] op);
  begin
    gm2 = {op[6:4], op[3]^op[7], op[2]^op[7], op[1], op[0]^op[7], op[7]}; //
  end
endfunction // gm2
////////////////////////////////////////////////////////////////////////////////


function [7 : 0] gm4(input [7 : 0] op);
  begin
    gm4 = gm2(gm2(op));
  end
endfunction // gm4

wire [7:0] w0, w1, w2, w3;    // data in
wire [7:0] a0, a1, a2, a3;    // mix data
wire [7:0] b0, b1, b2, b3;    // inv mix data

assign w0 = idata[31:24];
assign w1 = idata[23:16];
assign w2 = idata[15:08];
assign w3 = idata[07:00];

////////////////////////////////////////
wire [7:0] sum = w0 ^ w1 ^ w2 ^ w3;

assign a0 = gm2(w0 ^ w1) ^ sum ^ w0;
assign a1 = gm2(w1 ^ w2) ^ sum ^ w1;
assign a2 = gm2(w2 ^ w3) ^ sum ^ w2;
assign a3 = gm2(w3 ^ w0) ^ sum ^ w3;
////////////////////////////////////////


assign b0 = gm4(a0 ^ a2) ^ a0;
assign b1 = gm4(a1 ^ a3) ^ a1;
assign b2 = gm4(a0 ^ a2) ^ a2;
assign b3 = gm4(a1 ^ a3) ^ a3;

assign odata = (iinv) ? {b0, b1, b2, b3} : {a0, a1, a2, a3};

endmodule
```

3. Thực hiện tối ưu Key Schedule

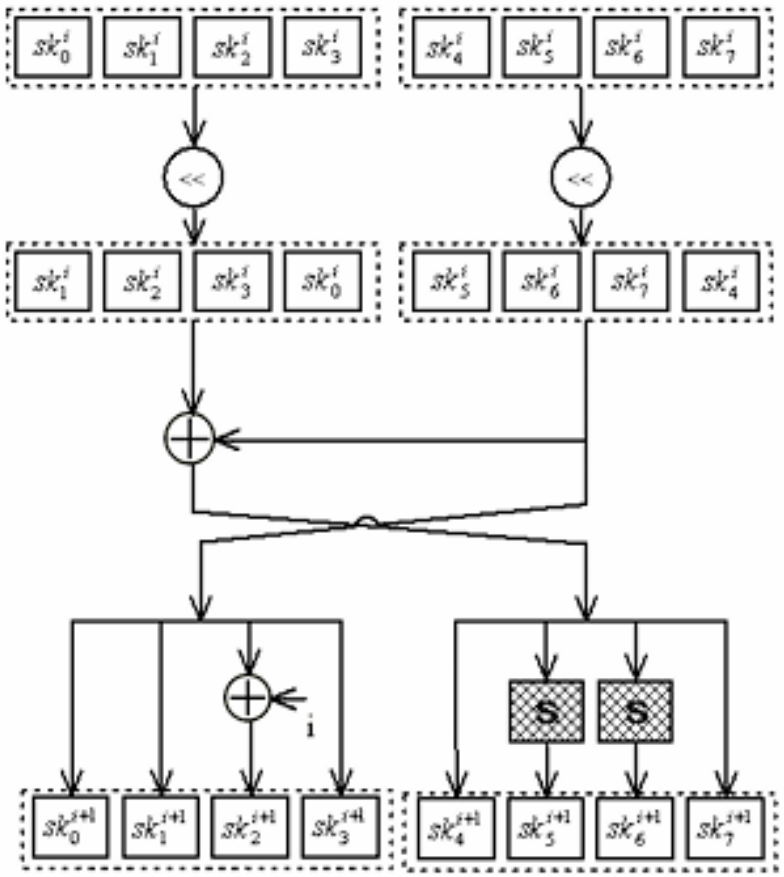
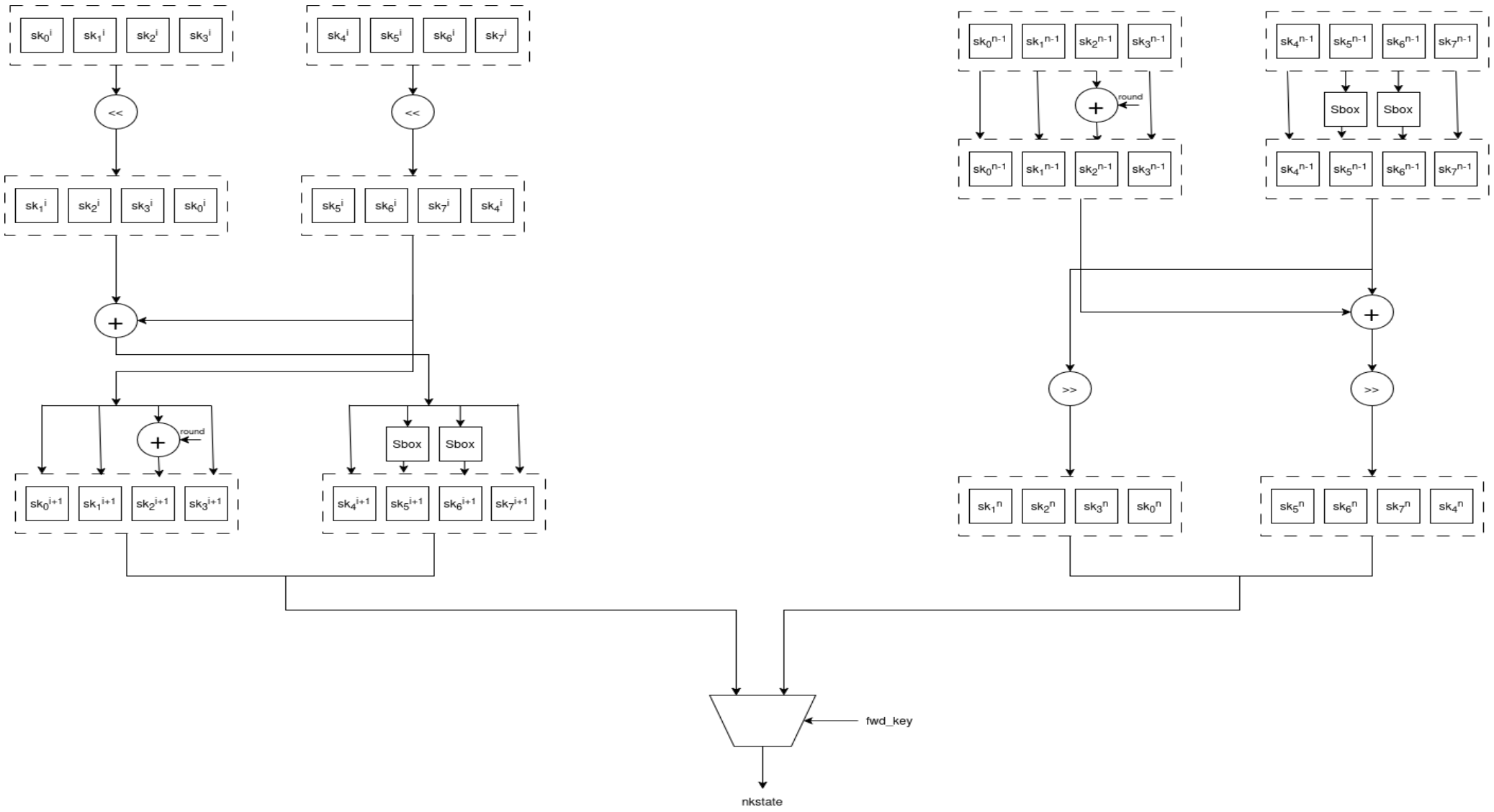


Figure 4: The KeySchedule algorithm of 64-bit key length.

1. Input: a 64/80/96-bit master key mk for KLEIN-64/80/96.
2. Key scheduling: Let i be the round counter of KLEIN-64/80/96. In the first round so that $i = 1$, the initial subkey $sk^1 = mk = sk_0^1 || sk_1^1 || \cdots || sk_t^1$ where $t = 7/9/11$ for KLEIN-64/80/96. For KLEIN-64, the $(i+1)$-th subkey sk^{i+1} can be derived from the i-th subkey sk^i as follows.
   (a) Divide the i-th subkey sk^i into two tuples, such that $a = (sk_0^i, sk_1^i, \cdots, sk_{\lfloor \frac{t}{2} \rfloor}^i)$ and $b = (sk_{\lceil \frac{t}{2} \rceil}^i, sk_{\lceil \frac{t}{2} \rceil + 1}^i, \cdots, sk_t^i)$ for the next step. For KlEIN-64, we have $a = (sk_0^i, sk_1^i, sk_2^i, sk_3^i)$ and $b = (sk_4^i, sk_5^i, sk_6^i, sk_7^i)$.
   (b) Cycling left shift one byte position in (a, b), obtain $a' = (sk_1^i, \cdots, sk_{\lfloor \frac{t}{2} \rfloor}^i, sk_0^i)$ and $b' = (sk_{\lceil \frac{t}{2} \rceil + 1}^i, \cdots, sk_t^i, sk_{\lceil \frac{t}{2} \rceil}^i)$ for the next step. For KLEIN-64, we have $a' = (sk_1^i, sk_2^i, sk_3^i, sk_0^i)$ and $b' = (sk_5^i, sk_6^i, sk_7^i, sk_4^i)$.
   (c) Swap the tuple (a', b') with a Feistel-like structure, such that $a'' = b'$ becomes the left tuple, whilst $b'' = a' \oplus b'$ becomes the right tuple.
   (d) Xor round counter i with the third byte in the left tuple a'', and substitute the second and the third bytes of the right tuple b'' by using the KLEIN S-box S.
3. Output: iteratively execute the above step for different key lengths, truncate the leftmost 64 bits of subkey sk^i for the i-th round transformation.

3. Thực hiện tối ưu Key Schedule



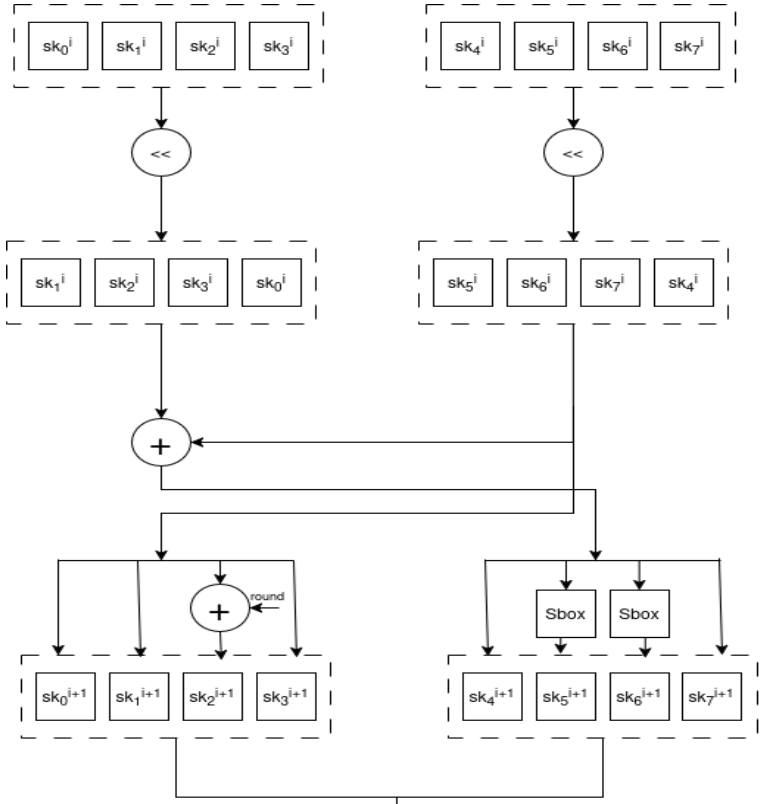
3. Thực hiện tối ưu Key Schedule

```rust
module Klein_KeySchedule (kstate, nkstate, round, is_fwd_key);
    input  wire [63:0] kstate;
    input  wire [3:0]  round;
    input  wire        is_fwd_key;

    output wire [63:0] nkstate;

    //STATE 1: Divide 2 tuples and Cycle left shift 1 byte (ENCRYPTION)
    wire [63:0] krot_enc = {kstate[55:32], kstate[63:56], kstate[23:0], kstate[31:24]};
    wire [63:0] kfei_enc;

    //STATE 2: Feistel-like structure (ENCRYPTION)
    assign kfei_enc[63:32] = krot_enc[31:00];
    assign kfei_enc[31:00] = krot_enc[63:32] ^ krot_enc[31:00];

    //STATE 3: S-box sk 5th and sk 6th (ENCRYPTION-DECRYPTION)
    wire [15:0] ks_sbox_in = is_fwd_key ? kfei_enc[23:8] : kstate[23:8];
    wire [15:0] ks_sbox_out;
    ////////////////////////////////////////////////////////////////
    Klein_Sbox sk0 (.in(ks_sbox_in[15:12]), .out(ks_sbox_out[15:12]));
    Klein_Sbox sk1 (.in(ks_sbox_in[11:08]), .out(ks_sbox_out[11:08]));
    ////////////////////////////////////////////////////////////////
    Klein_Sbox sk2 (.in(ks_sbox_in[7:4]),   .out(ks_sbox_out[7:4]));
    Klein_Sbox sk3 (.in(ks_sbox_in[3:0]),   .out(ks_sbox_out[3:0]));

     //STATE 3: (ENCRYPTION)
    wire [63:0] nkstate_enc;
    assign nkstate_enc[63:48] = kfei_enc[63:48];
    assign nkstate_enc[47:40] = kfei_enc[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter i

    assign nkstate_enc[39:24] = kfei_enc[39:24];
    assign nkstate_enc[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
    assign nkstate_enc[7:0]   = kfei_enc[7:0];

   //STATE 1: (DECRYPTION)
    wire [63:0] kfei_dec_pre;
    assign kfei_dec_pre[63:48] = kstate[63:48];
    assign kfei_dec_pre[47:40] = kstate[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter i
    assign kfei_dec_pre[39:24] = kstate[39:24];
    assign kfei_dec_pre[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
    assign kfei_dec_pre[7:0]   = kstate[7:0];

   //STATE 2: Feistel-like structure (DECRYPTION)
    wire [63:0] krot_dec;
    assign krot_dec[31:0]  = kfei_dec_pre[63:32];
    assign krot_dec[63:32] = kfei_dec_pre[63:32] ^ kfei_dec_pre[31:0];

    //STATE 3: Rotate (DECRYPTION)
    wire [63:0] nkstate_dec = {krot_dec[39:32], krot_dec[63:40], krot_dec[7:0], krot_dec[31:8]};

    //MUX : ENCRYPTION AND DECRYPTION
    assign nkstate = is_fwd_key ? nkstate_enc : nkstate_dec;
endmodule
```

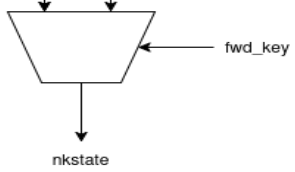
```rust
//STATE 1: Divide 2 tuples and Cycle left shift 1 byte (ENCRYPTION)
wire [63:0] krot_enc = {kstate[55:32], kstate[63:56], kstate[23:0], kstate[31:24]};
wire [63:0] kfei_enc;

//STATE 2: Feistel-like structure (ENCRYPTION)
assign kfei_enc[63:32] = krot_enc[31:00];
assign kfei_enc[31:00] = krot_enc[63:32] ^ krot_enc[31:00];

//STATE 3: S-box sk 5th and sk 6th (ENCRYPTION-DECRYPTION)
wire [15:0] ks_sbox_in = is_fwd_key ? kfei_enc[23:8] : kstate[23:8];
wire [15:0] ks_sbox_out;
////////////////////////////////////////////////////////////////
Klein_Sbox sk0 (.in(ks_sbox_in[15:12]), .out(ks_sbox_out[15:12]));
Klein_Sbox sk1 (.in(ks_sbox_in[11:08]), .out(ks_sbox_out[11:08]));
////////////////////////////////////////////////////////////////
Klein_Sbox sk2 (.in(ks_sbox_in[7:4]),   .out(ks_sbox_out[7:4]));
Klein_Sbox sk3 (.in(ks_sbox_in[3:0]),   .out(ks_sbox_out[3:0]));

 //STATE 3: (ENCRYPTION)
wire [63:0] nkstate_enc;
assign nkstate_enc[63:48] = kfei_enc[63:48];
assign nkstate_enc[47:40] = kfei_enc[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter i

assign nkstate_enc[39:24] = kfei_enc[39:24];
assign nkstate_enc[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
assign nkstate_enc[7:0]   = kfei_enc[7:0];
```

```rust
//STATE 1: (DECRYPTION)
 wire [63:0] kfei_dec_pre;
 assign kfei_dec_pre[63:48] = kstate[63:48];
 assign kfei_dec_pre[47:40] = kstate[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter i
 assign kfei_dec_pre[39:24] = kstate[39:24];
 assign kfei_dec_pre[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
 assign kfei_dec_pre[7:0]   = kstate[7:0];

//STATE 2: Feistel-like structure (DECRYPTION)
 wire [63:0] krot_dec;
 assign krot_dec[31:0]  = kfei_dec_pre[63:32];
 assign krot_dec[63:32] = kfei_dec_pre[63:32] ^ kfei_dec_pre[31:0];

 //STATE 3: Rotate (DECRYPTION)
 wire [63:0] nkstate_dec = {krot_dec[39:32], krot_dec[63:40], krot_dec[7:0], krot_dec[31:8]};

 //MUX : ENCRYPTION AND DECRYPTION
```

3. Thực hiện tối ưu Key Schedule

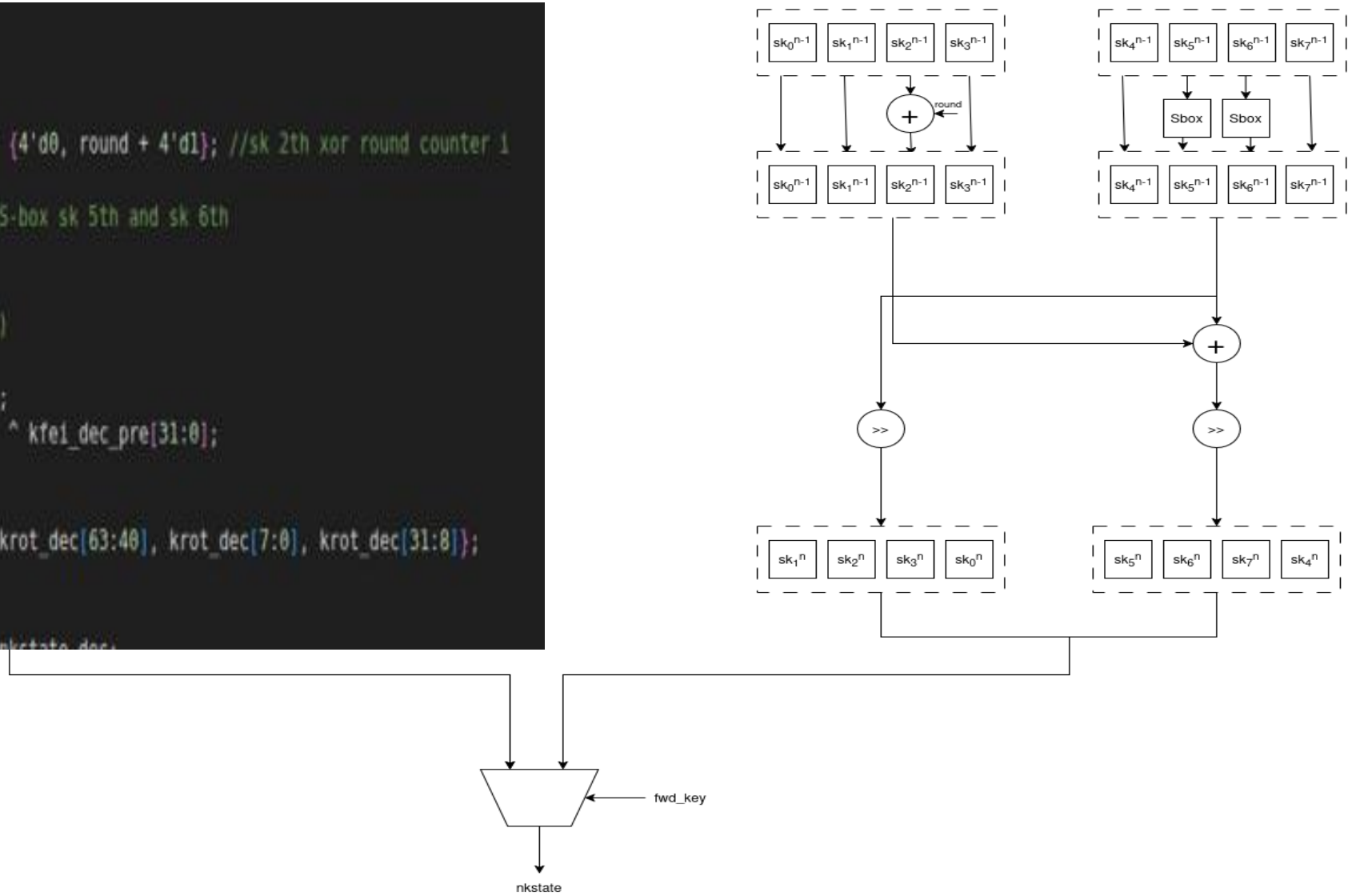



```
//STATE 1: Divide 2 tuples and Cycle left shift 1 byte (ENCRYPTION)
wire [63:0] krot_enc = {kstate[55:32], kstate[63:56], kstate[23:0], kstate[31:24]};
wire [63:0] kfei_enc;

//STATE 2: Feistel-like structure (ENCRYPTION)
assign kfei_enc[63:32] = krot_enc[31:00];
assign kfei_enc[31:00] = krot_enc[63:32] ^ krot_enc[31:00];

//STATE 3: S-box sk 5th and sk 6th (ENCRYPTION-DECRYPTION)
wire [15:0] ks_sbox_in = is_fwd_key ? kfei_enc[23:8] : kstate[23:8];
wire [15:0] ks_sbox_out;
//////////////////////////////////////////////////////////////////
Klein_Sbox sk0 (.in(ks_sbox_in[15:12]), .out(ks_sbox_out[15:12]));
Klein_Sbox sk1 (.in(ks_sbox_in[11:08]), .out(ks_sbox_out[11:08]));
//////////////////////////////////////////////////////////////////
Klein_Sbox sk2 (.in(ks_sbox_in[7:4]),   .out(ks_sbox_out[7:4]));
Klein_Sbox sk3 (.in(ks_sbox_in[3:0]),   .out(ks_sbox_out[3:0]));

 //STATE 3: (ENCRYPTION)
wire [63:0] nkstate_enc;
assign nkstate_enc[63:48] = kfei_enc[63:48];
assign nkstate_enc[47:40] = kfei_enc[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter 1

assign nkstate_enc[39:24] = kfei_enc[39:24];
assign nkstate_enc[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
assign nkstate_enc[7:0]   = kfei_enc[7:0];
```



3. Thực hiện tối ưu Key Schedule

```
//STATE 1: (DECRYPTION)
 wire [63:0] kfei_dec_pre;
 assign kfei_dec_pre[63:48] = kstate[63:48];
 assign kfei_dec_pre[47:40] = kstate[47:40] ^ {4'd0, round + 4'd1}; //sk 2th xor round counter 1
 assign kfei_dec_pre[39:24] = kstate[39:24];
 assign kfei_dec_pre[23:8]  = ks_sbox_out; //S-box sk 5th and sk 6th
 assign kfei_dec_pre[7:0]   = kstate[7:0];

//STATE 2: Feistel-like structure (DECRYPTION)
 wire [63:0] krot_dec;
 assign krot_dec[31:0]  = kfei_dec_pre[63:32];
 assign krot_dec[63:32] = kfei_dec_pre[63:32] ^ kfei_dec_pre[31:0];

 //STATE 3: Rotate (DECRYPTION)
 wire [63:0] nkstate_dec = {krot_dec[39:32], krot_dec[63:40], krot_dec[7:0], krot_dec[31:8]};

 //MUX : ENCRYPTION AND DECRYPTION
```



2. Structure of KLEIN

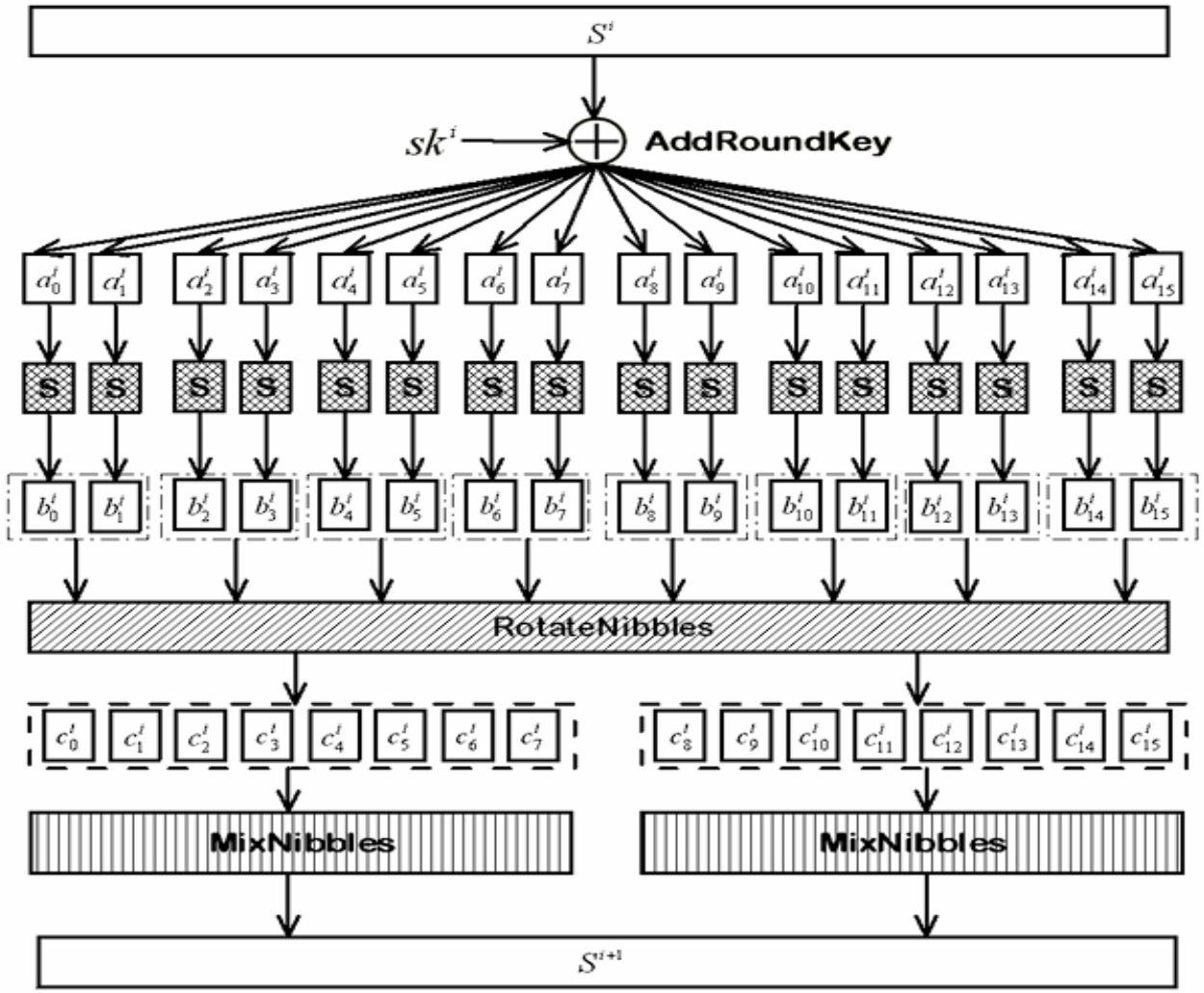
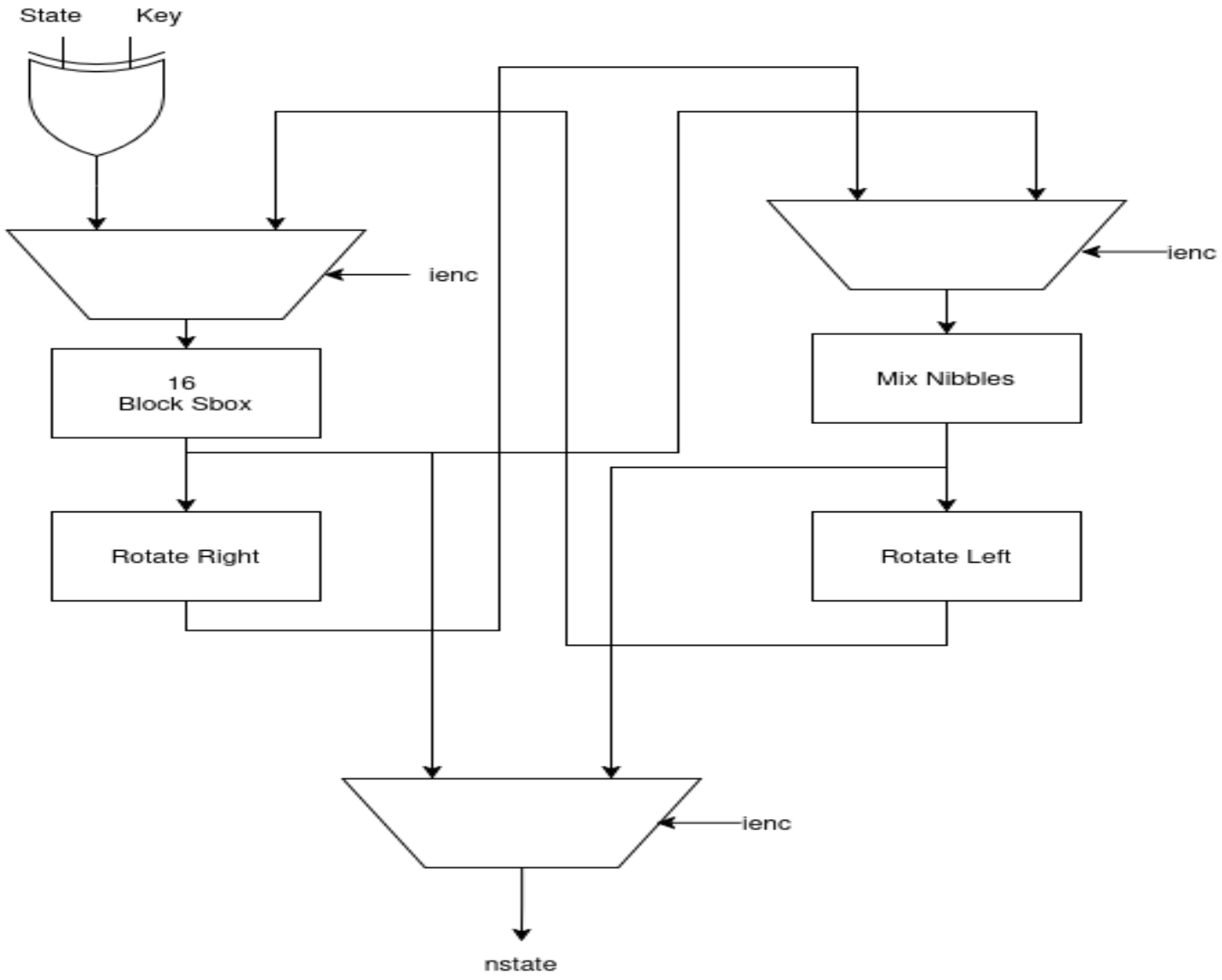
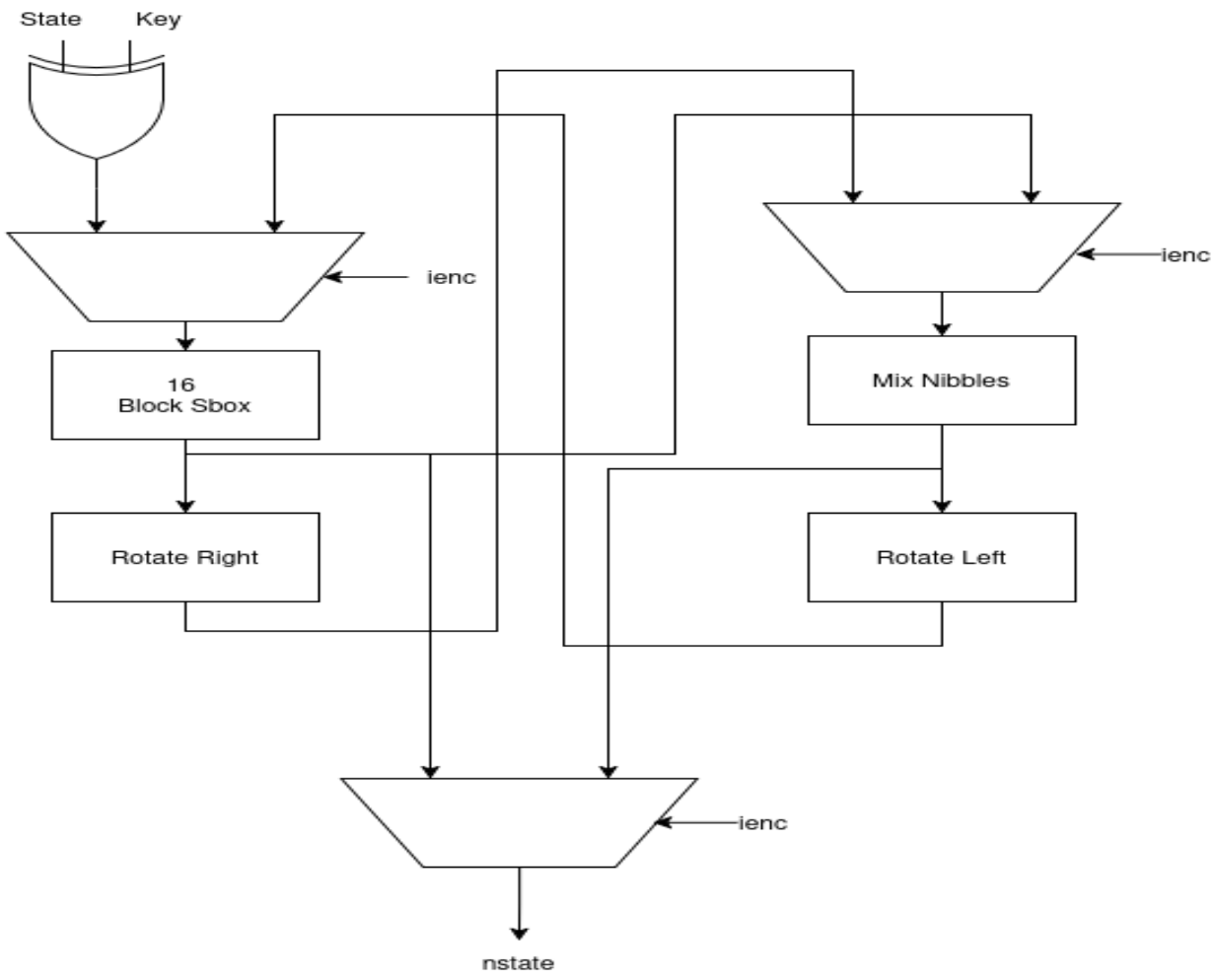


Figure 2: The round transformation of KLEIN.

4. Thực hiện tối ưu Datapath

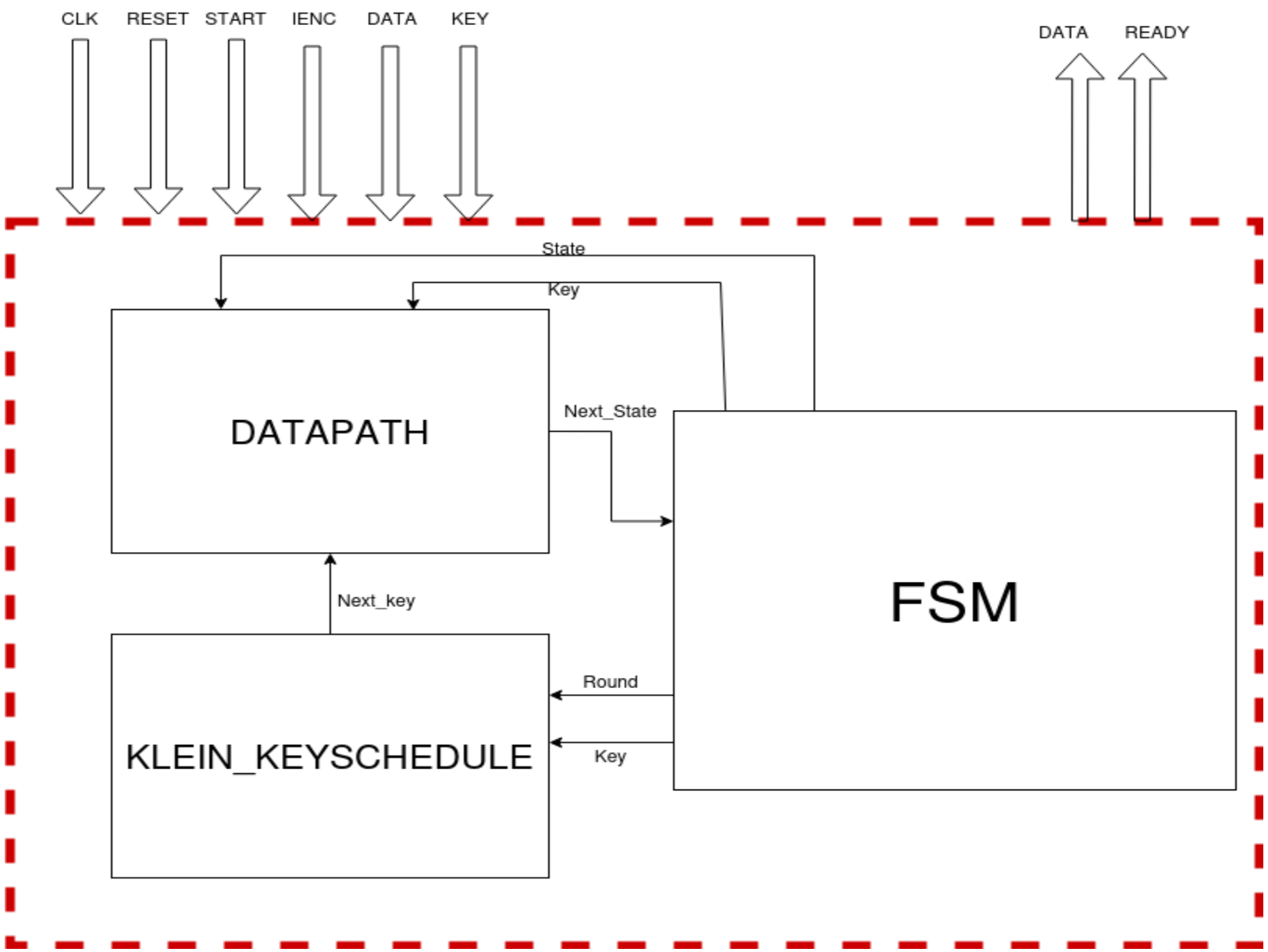


4. Thực hiện tối ưu Datapath




```
module Klein_Datapath (
    input  wire [63:0] state,
    input  wire [63:0] kstate,
    input  wire        ienc,
    output wire [63:0] nstate
);

    wire [63:0] ssum = state ^ kstate;


    wire [63:0] srot_dec;

    wire [63:0] sbox_in = ienc ? ssum : srot_dec;
    wire [63:0] sbox_out;

    Klein_Sbox s1  (.in(sbox_in[63:60]), .out(sbox_out[63:60]));
    Klein_Sbox s2  (.in(sbox_in[59:56]), .out(sbox_out[59:56]));
    Klein_Sbox s3  (.in(sbox_in[55:52]), .out(sbox_out[55:52]));
    Klein_Sbox s4  (.in(sbox_in[51:48]), .out(sbox_out[51:48]));
    Klein_Sbox s5  (.in(sbox_in[47:44]), .out(sbox_out[47:44]));
    Klein_Sbox s6  (.in(sbox_in[43:40]), .out(sbox_out[43:40]));
    Klein_Sbox s7  (.in(sbox_in[39:36]), .out(sbox_out[39:36]));
    Klein_Sbox s8  (.in(sbox_in[35:32]), .out(sbox_out[35:32]));
    Klein_Sbox s9  (.in(sbox_in[31:28]), .out(sbox_out[31:28]));
    Klein_Sbox s10 (.in(sbox_in[27:24]), .out(sbox_out[27:24]));
    Klein_Sbox s11 (.in(sbox_in[23:20]), .out(sbox_out[23:20]));
    Klein_Sbox s12 (.in(sbox_in[19:16]), .out(sbox_out[19:16]));
    Klein_Sbox s13 (.in(sbox_in[15:12]), .out(sbox_out[15:12]));
    Klein_Sbox s14 (.in(sbox_in[11:08]), .out(sbox_out[11:08]));
    Klein_Sbox s15 (.in(sbox_in[07:04]), .out(sbox_out[07:04]));
    Klein_Sbox s16 (.in(sbox_in[03:00]), .out(sbox_out[03:00]));


    wire [63:0] srot_enc = {sbox_out[47:0], sbox_out[63:48]};

    wire [63:0] mix_in = ienc ? srot_enc : ssum;
    wire [63:0] mix_out;

    Klein_Mixnibbles mix1(.idata(mix_in[63:32]), .iinv(!ienc), .odata(mix_out[63:32]));
    Klein_Mixnibbles mix2(.idata(mix_in[31:00]), .iinv(!ienc), .odata(mix_out[31:00]));

    assign srot_dec = {mix_out[15:0], mix_out[63:16]};

    assign nstate = ienc ? mix_out : sbox_out;

endmodule
```

4. Thực hiện tối ưu Core



4. Thực hiện tối ưu Core

```
module Klein_Full_Core (iclk, ireset, istart, iblock, ienc, ikey, oready, oblock);
    input   wire          iclk;
    input   wire          ireset;
    input   wire          istart;
    input   wire          ienc;
    input   wire  [63:0]  iblock;
    input   wire  [63:0]  ikey;

    output  wire          oready;
    output  wire  [63:0]  oblock;

    localparam S_IDLE  = 2'd0;
    localparam S_SETUP = 2'd1;
    localparam S_RUN   = 2'd2;

    reg [1:0]   fsm_state;
    reg         ready_reg;
    reg [63:0]  result_reg;
    reg [3:0]   round;
    reg [63:0]  state, kstate;

    reg [63:0]  cached_k0;
    reg [63:0]  cached_k12;
    reg         k12_valid;

    wire [63:0] nstate, nkstate;

    assign oready = ready_reg;
    assign oblock = result_reg;

    Klein_Datapath datapath (
        .state  (state),
        .kstate (kstate),
        .ienc   (ienc),
        .nstate (nstate)
    );


    wire is_fwd_key = ienc | (fsm_state == S_SETUP);
    Klein_KeySchedule keyschedule (.kstate(kstate), .round(round), .is_fwd_key(is_fwd_key), .nkstate(nkstate));

    always @(posedge iclk) begin
        if (ireset) begin
            fsm_state   <= S_IDLE;
            ready_reg   <= 1'b0;
            result_reg  <= 64'd0;
            round       <= 4'd0;
            state       <= 64'd0;
            kstate      <= 64'd0;
            cached_k0   <= 64'd0;
            cached_k12  <= 64'd0;
            k12_valid   <= 1'b0;
        end
        else begin
            case (fsm_state)
                S_IDLE: begin
                    ready_reg <= 1'b0;
                    if (istart) begin
                        state <= iblock;
                        if (ikey != cached_k0 || !k12_valid) begin
                            cached_k0 <= ikey;
                            k12_valid <= 1'b0;
                            if (!ienc) begin
                                fsm_state <= S_SETUP;
```

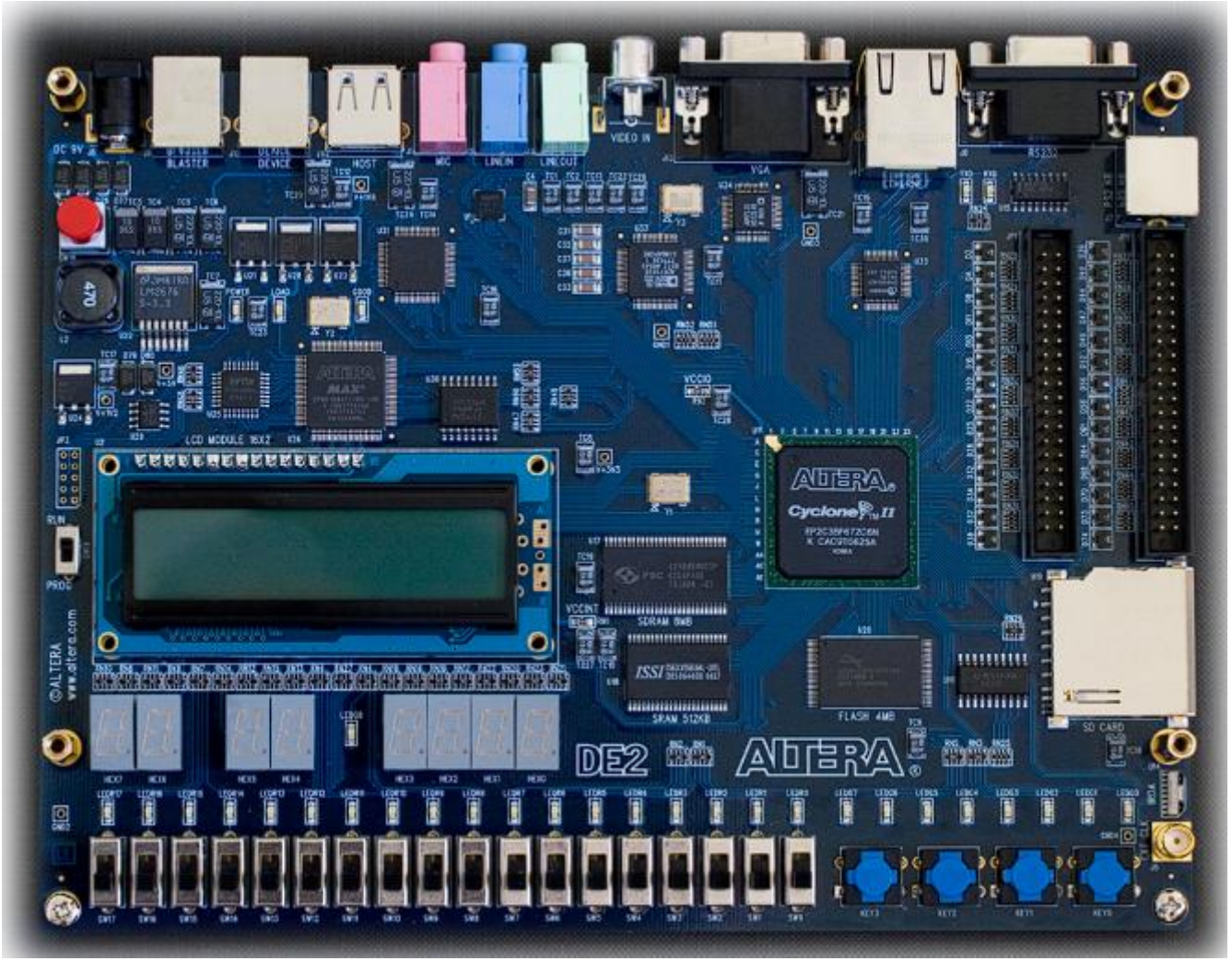
```
                                round     <= 4'd0;
                                kstate    <= ikey;
                            end else begin
                                fsm_state <= S_RUN;
                                round     <= 4'd0;
                                kstate    <= ikey;
                            end
                        end
                        else begin
                            fsm_state <= S_RUN;
                            if (ienc) begin
                                round  <= 4'd0;
                                kstate <= ikey;
                            end else begin
                                round  <= 4'd11;
                                kstate <= cached_k12;
                            end
                        end
                    end
                end

                S_SETUP: begin
                    if (round < 4'd11) begin
                        round  <= round + 4'd1;
                        kstate <= nkstate;
                    end
                    else begin
                        cached_k12 <= nkstate;
                        k12_valid  <= 1'b1;
                        fsm_state  <= S_RUN;
                        round      <= 4'd11;
                        kstate     <= nkstate;
                    endz
                end

                S_RUN: begin
                    state  <= nstate;
                    kstate <= nkstate;

                    if (ienc) begin
                        if (round < 4'd11) round <= round + 4'd1;
                        else begin
                            result_reg <= nstate ^ nkstate;
                            cached_k12 <= nkstate;
                            k12_valid  <= 1'b1;
                            ready_reg  <= 1'b1;
                            fsm_state  <= S_IDLE;
                        end
                    end
                    else begin
                        if (round > 4'd0) round <= round - 4'd1;
                        else begin
                            result_reg <= nstate ^ nkstate;
                            ready_reg  <= 1'b1;
                            fsm_state  <= S_IDLE;
                        end
                    end
                end
            endcase
        end
    end
endmodule
```

4. Thực hiện Testbench

```
`timescale 1ns / 1ps

module tb_Klein_Core;

    parameter CLK_HALF_PERIOD = 5;
    parameter CLK_PERIOD      = 2 * CLK_HALF_PERIOD;

    reg         tb_iclk;
    reg         tb_ireset;
    reg         tb_istart;
    reg         tb_ienc;
    reg  [63:0] tb_iblock;
    reg  [63:0] tb_ikey;

    wire        tb_oready;
    wire [63:0] tb_oblock;

    Klein_Full_Core dut (
        .iclk(tb_iclk),
        .ireset(tb_ireset),
        .istart(tb_istart),
        .ienc(tb_ienc),
        .iblock(tb_iblock),
        .ikey(tb_ikey),
        .oready(tb_oready),
        .oblock(tb_oblock)
    );


    always #CLK_HALF_PERIOD tb_iclk = ~tb_iclk;

    task run_test;
        input [63:0]  t_key;
        input [63:0]  t_in;
        input [63:0]  t_expected;
        input         t_is_enc;

        begin
            tb_ikey   = t_key;
            tb_iblock = t_in;
            tb_ienc   = t_is_enc;

            tb_istart = 1'b1;
            #(CLK_PERIOD);
            tb_istart = 1'b0;

            wait(tb_oready);

            if (tb_oblock === t_expected) begin
                $display("[PASS] %s", t_is_enc ? "ENCRYPT" : "DECRYPT");
                $display("   Key : %16h | In : %16h | Out : %16h", t_key, t_in, tb_oblock);
            end else begin
                $display("[FAIL] %s", t_is_enc ? "ENCRYPT" : "DECRYPT");
                $display("   Key : %16h | In : %16h", t_key, t_in);
                $display("   Exp : %16h | Got: %16h  <-- ERROR", t_expected, tb_oblock);
            end
            $display("\n");

            #(3 * CLK_PERIOD);
        end
    endtask


    initial begin
        tb_iclk   = 0;
        tb_ireset = 1;
        tb_istart = 0;
        tb_ienc   = 1;
        tb_iblock = 64'd0;
        tb_ikey   = 64'd0;

        #(4 * CLK_PERIOD);
        tb_ireset = 0;
        #(2 * CLK_PERIOD);


        // --- VECTOR 1 ---
        run_test(64'h0000000000000000, 64'hFFFFFFFFFFFFFFFF, 64'hCDC0B51F14722BBE, 1); // ENCRYPTION
        run_test(64'h0000000000000000, 64'hCDC0B51F14722BBE, 64'hFFFFFFFFFFFFFFFF, 0); // DECRYPTION

        // --- VECTOR 2 ---
        run_test(64'hFFFFFFFFFFFFFFFF, 64'h0000000000000000, 64'h6456764E8602E154, 1); // ENCRYPTION
        run_test(64'hFFFFFFFFFFFFFFFF, 64'h6456764E8602E154, 64'h0000000000000000, 0); // DECRYPTION

        // --- VECTOR 3 ---
        run_test(64'h1234567890ABCDEF, 64'hFFFFFFFFFFFFFFFF, 64'h592356C4997176C8, 1); // ENCRYPTION
        run_test(64'h1234567890ABCDEF, 64'h592356C4997176C8, 64'hFFFFFFFFFFFFFFFF, 0); // DECRYPTION

        // --- VECTOR 4 ---
        run_test(64'h0000000000000000, 64'h1234567890ABCDEF, 64'h629F9D6DFF95800E, 1); // ENCRYPTION
        run_test(64'h0000000000000000, 64'h629F9D6DFF95800E, 64'h1234567890ABCDEF, 0); // DECRYPTION

        $finish;
    end

endmodule
```

```
[PASS] ENCRYPT
   Key : 0000000000000000 | In : ffffffffffffffff | Out : cdc0b51f14722bbe


[PASS] DECRYPT
   Key : 0000000000000000 | In : cdc0b51f14722bbe | Out : ffffffffffffffff


[PASS] ENCRYPT
   Key : ffffffffffffffff | In : 0000000000000000 | Out : 6456764e8602e154


[PASS] DECRYPT
   Key : ffffffffffffffff | In : 6456764e8602e154 | Out : 0000000000000000


[PASS] ENCRYPT
   Key : 1234567890abcdef | In : ffffffffffffffff | Out : 592356c4997176c8


[PASS] DECRYPT
   Key : 1234567890abcdef | In : 592356c4997176c8 | Out : ffffffffffffffff


[PASS] ENCRYPT
   Key : 0000000000000000 | In : 1234567890abcdef | Out : 629f9d6dff95800e


[PASS] DECRYPT
   Key : 0000000000000000 | In : 629f9d6dff95800e | Out : 1234567890abcdef
```

4. Chuẩn bị Nạp Board DE2



4. Dự định Tìm hiểu FIPS 205

FIPS 205

Federal Information Processing Standards Publication

Stateless Hash-Based Digital Signature Standard

Category: Computer Security **Subcategory: Cryptography**

Information Technology Laboratory
National Institute of Standards and Technology
Gaithersburg, MD 20899-8900

This publication is available free of charge from:
https://doi.org/10.6028/NIST.FIPS.205

Published: August 13, 2024

fetel@HCMUS
KHOA ĐIỆN TỬ - VIỄN THÔNG

DESLAB

THANK YOU!